

Log-File Analysis Web-App (OnlyFiles)

Aesthetic & powerful visualizations!

Nitanshu

24b0974

April 29, 2025

Abstract

This report explores the design, implementation, and features of a web-based log file analysis tool. The project combines the speed and reliability of Bash scripting for parsing and converting log files into CSV, with the flexibility and power of Python (using Flask, pandas, and matplotlib) to deliver an interactive web interface and rich visualizations. The journey of building this application was anything but ordinary-coding sessions often took place against the backdrop of Goa's beaches, and progress was regularly challenged by the irresistible call of the sea and the constant, well-meaning interruptions from my friend Nipun.

Contents

1	Introduction	2
2	Requirements	2
2.1	Software and Modules	2
2.2	Installation Commands	2
3	Running Instructions	2
4	Website Layout	3
5	Modules Used	3
6	Directory Structure and File Descriptions	3
7	Features	5
7.1	Basic Features	5
7.2	Advanced Features	6
8	Project Journey	7
9	Bibliography	8

1 Introduction

Log files are the irreplaceable - from system monitoring, debugging to security. Yet, trawling through endless lines of raw log data is about as fun as debugging JavaScript at 3am. This project aims to make log analysis not only efficient but also visually insightful—and, dare I say, a little bit fun. The application lets users upload log files, parses them into structured CSV using Bash, and then provides a web interface for filtering, sorting, and visualizing log data.

2 Requirements

2.1 Software and Modules

- **Python**
- **Flask** (web framework)
- **pandas** (data handling)
- **matplotlib** (visualizations)
- **numpy** (support for matplotlib/pandas)
- **Bash** (for log parsing and CSV conversion)

2.2 Installation Commands

Assuming you have Python and Bash:

```
# Create and activate a virtual environment (optional but recommended)
python3 -m venv venv
source venv/bin/activate
```

```
# Install required Python modules
pip install flask pandas matplotlib numpy
```

3 Running Instructions

1. **Clone or download** the project directory.
2. **Place your Apache log files** in a known location.
3. **Activate the virtual environment:**

```
source venv/bin/activate
```

4. **Run the Flask app:**

```
python app.py
```

5. **Open your browser** and visit `http://127.0.0.1:5000/upload`

4 Website Layout

The website is structured into three main pages:

- **Log Upload Page:** Upload Page — Users can drag-and-drop or select a log file. Filtering and sorting options are available *before* upload. **Upon submission you are redirected to a page which displays the amount of lines processed and total lines**, logs are parsed by a Bash script and converted to CSV.
- **Log Display Page:** Display Page — Shows the parsed logs in a table, **also mentions the lines that were of the wrong format and skipped**, gives a button to display graphs which leads to the graph page. Users can download the CSV or re-upload a new log. No filtering/sorting is done here—that’s all Bash, baby.
- **Graphs & Plots Page:** Graphs Page — Displays a suite of visualizations (line plot, pie chart, bar chart, stacked bar, heatmap, histogram, scatter, area chart). Users can also run their own custom Python code for bespoke plots, right in the browser.

You may have to reload this page once for the graphs to be rendered properly, also note that in the graph page, the data has already been filtered by the original filters in the upload page.

5 Modules Used

- **Flask:** Web server and routing.
- **pandas:** DataFrame operations for CSV and plotting.
- **matplotlib:** All visualizations (line, pie, bar, heatmap, etc.).
- **numpy:** Array and numerical support for pandas/matplotlib.
- **Bash:** All log parsing, filtering, and CSV conversion.

6 Directory Structure and File Descriptions

The project directory is organized as follows:

```
project-root/  
  
app.py  
visualization.py  
scripts/  
    process_logs.sh  
static/  
    css/  
        style.css  
    js/  
        upload.js
```

```
    display.js
    processing.js
    graph.js
images/
templates/
    base.html
    upload.html
    display.html
    graph.html
    processing.html
```

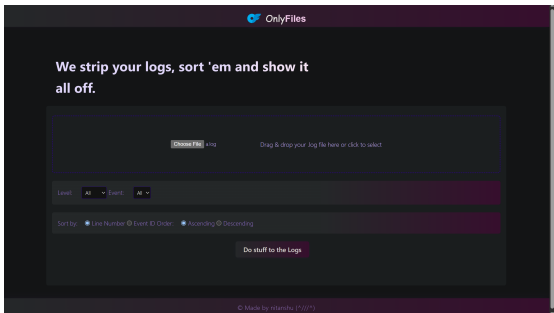
- **app.py**: The main Flask web application. Handles all routing, user requests, file uploads, progress polling, and rendering of all HTML pages. It launches the Bash script for log parsing and passes user filter/sort options. Also manages the execution of user custom code for generating plots.
- **visualization.py**: Contains all logic for loading the parsed CSV file as a pandas DataFrame and generating the suite of visualizations (line plot, pie chart, bar chart, heatmap, etc.) using matplotlib. Also provides a function to execute custom user plotting code.
- **scripts/process_logs.sh**: A Bash script responsible for parsing the uploaded log file, filtering by level or event, sorting as requested, and converting the result to a structured CSV. It also keeps track of malformed/skipped lines for user feedback.
- **static/css/style.css**: Contains all CSS styling for the website, including layout, color themes, responsive design, and UI components like headers, footers, tables, buttons, and forms.
- **static/js/upload.js**: Handles drag-and-drop file uploads and user interaction on the upload page.
- **static/js/graph.js**: Handles download button actions and any interactive features on the graphs page.
- **static/images/**: Stores all generated plot images (PNG format) for display and download.
- **templates/base.html**: The base Jinja2 template for all pages, providing the shared header, footer, and layout structure.
- **templates/upload.html**: The log upload page. Contains the drag-and-drop upload form, filter and sort options, and a motivational quote.
- **templates/display.html**: Displays the parsed logs in a table, shows any skipped/malformed lines, and provides buttons for re-upload, download, and accessing graphs.
- **templates/graph.html**: Shows all generated plots and includes the embedded Python code editor for custom user visualizations.
- **templates/processing.html**: Shows a progress bar and parsing status while the Bash script processes the log file.

This structure cleanly separates backend logic, static assets, and HTML templates, making the project easy to maintain and extend.

7 Features

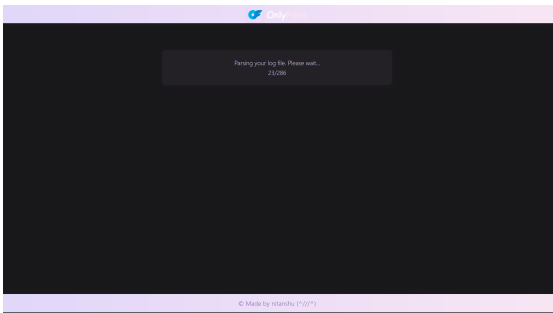
This project provides a robust set of features to make log analysis fast, accurate, and visually insightful. Features are grouped as basic and advanced for clarity, with each screenshot placed just before the feature it illustrates. Two features (and their screenshots) are shown per page for a balanced layout.

7.1 Basic Features



Log upload with drag-and-drop:

The upload page allows users to drag and drop their log file or select it manually, making the process simple and user-friendly. Filtering and sorting options are available before upload.



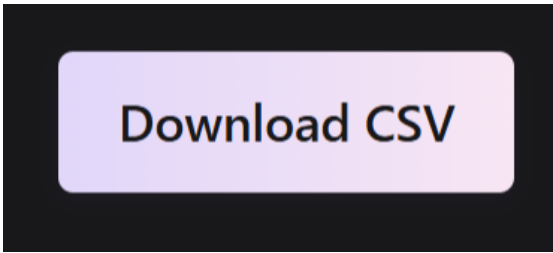
Live progress bar:

While the log file is being parsed, users see a real-time progress indicator (e.g., 184/240), so they always know how much work remains.

Line ID	Timestamp	Level	Content	Event ID	Event Template
1	Sun Dec 04 04:47:44 2005	notice	worker[envrnt] ok /etc/httpd/conf/worker2.properties	E2	worker[envrnt] ok <*>
2	Sun Dec 04 04:47:44 2005	error	mod_jk child worker[env] in error state 6	E3	mod_jk child worker[env] in error state <*>
3	Sun Dec 04 04:51:08 2005	notice	j2_maj Found child 6725 in scoreboard slot 10	E1	j2_maj Found child <*> in scoreboard slot <*>
4	Sun Dec 04 04:51:09 2005	notice	j2_maj Found child 6726 in scoreboard slot 8	E1	j2_maj Found child <*> in scoreboard slot <*>
5	Sun Dec 04 04:51:09 2005	notice	j2_maj Found child 6728 in scoreboard slot 6	E1	j2_maj Found child <*> in scoreboard slot <*>
6	Sun Dec 04 04:51:14 2005	notice	worker[envrnt] ok /etc/httpd/conf/worker2.properties	E2	worker[envrnt] ok <*>
7	Sun Dec 04 04:51:14 2005	notice	worker[envrnt] ok /etc/httpd/conf/worker2.properties	E2	worker[envrnt] ok <*>

Table view:

After parsing, all logs are displayed in a clean, scrollable table. Each row corresponds to a log entry, with columns for every field.



CSV download:

Users can download the processed and structured log data as a CSV file with a single click.

```
# Write CSV header
echo "Line ID,Timestamp,Level,Content,Event ID,Event Template" > "$TMP_F1"

# Parse one log line, output CSV if valid, else return 1
process_line() {
    local line="$1"
    local LINE_ID="$2"
    [[ -z "$line" ]] && return 1
    if [[ ! "$line" =~ ^\[.*\]\ \[.*\]\ .* ]]; then
        return 1
    fi
    local TIMESTAMP=$(grep -oP '\[.*\]' << "$line")
    local LEVEL=$(grep -oP '\[.*\]' << "$line")
    local CONTENT=$(sed -E 's/\[.*\]\ \[.*\] //' << "$line")
    local EVENT_ID="Unknown"
    local EVENT_TEMPLATE="$CONTENT"
    case "$CONTENT" in
        "jk2_init() Found child*" ) EVENT_ID="E1"; EVENT_TEMPLATE="jk2_init()"
        "workerEnv.init() ok*" ) EVENT_ID="E2"; EVENT_TEMPLATE="workerEnv.init()"
        "mod_jk child workerEnv in error state*" ) EVENT_ID="E3"; EVENT_TEMPLATE="mod_jk child workerEnv in error state"
        "[client*" Directory index forbidden by rule*" ) EVENT_ID="E4"; EVENT_TEMPLATE="client*"
        "jk2_init() Can't find child*" ) EVENT_ID="E5"; EVENT_TEMPLATE="jk2_init() Can't find child"
        "mod_jk child init*" ) EVENT_ID="E6"; EVENT_TEMPLATE="mod_jk child init"
    esac
}

# Apply filters if set
if [[ -n "$FILTER_LEVEL" && "$LEVEL" != "$FILTER_LEVEL" ]]; then
```

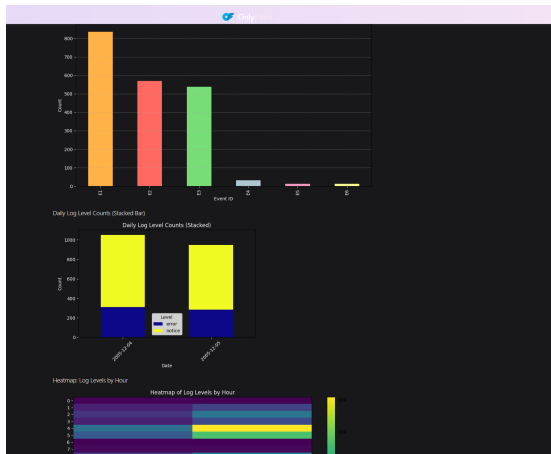
Bash-powered parsing:

All parsing, filtering, and sorting is handled by a Bash script, ensuring fast and reliable processing even for large log files.

Line ID	Timestamp	Level	Content	Event ID	Event Template
1	Sun Dec 04	notice	workerEnv: E2	workerEnv.init() ok <*>	
2	Sun Dec 04	error	mod_jk chi E3	mod_jk child workerEnv in error state <*>	
3	Sun Dec 04	notice	jk2_init() F: E1	jk2_init() Found child <*> in scoreboard slot <*>	
4	Sun Dec 04	notice	jk2_init() F: E1	jk2_init() Found child <*> in scoreboard slot <*>	
5	Sun Dec 04	notice	jk2_init() F: E1	jk2_init() Found child <*> in scoreboard slot <*>	
6	Sun Dec 04	notice	workerEnv: E2	workerEnv.init() ok <*>	
7	Sun Dec 04	notice	workerEnv: E2	workerEnv.init() ok <*>	
8	Sun Dec 04	notice	workerEnv: E2	workerEnv.init() ok <*>	
9	Sun Dec 04	error	mod_jk chi E3	mod_jk child workerEnv in error state <*>	
10	Sun Dec 04	error	mod_jk chi E3	mod_jk child workerEnv in error state <*>	
11	Sun Dec 04	error	mod_jk chi E3	mod_jk child workerEnv in error state <*>	
12	Sun Dec 04	notice	jk2_init() F: E1	jk2_init() Found child <*> in scoreboard slot <*>	
13	Sun Dec 04	notice	jk2_init() F: E1	jk2_init() Found child <*> in scoreboard slot <*>	
14	Sun Dec 04	notice	jk2_init() F: E1	jk2_init() Found child <*> in scoreboard slot <*>	
15	Sun Dec 04	notice	workerEnv: E2	workerEnv.init() ok <*>	
16	Sun Dec 04	notice	workerEnv: E2	workerEnv.init() ok <*>	
17	Sun Dec 04	error	mod_jk chi E3	mod_jk child workerEnv in error state <*>	
18	Sun Dec 04	notice	jk2_init() F: E1	jk2_init() Found child <*> in scoreboard slot <*>	
19	Sun Dec 04	notice	jk2_init() F: E1	jk2_init() Found child <*> in scoreboard slot <*>	
20	Sun Dec 04	notice	jk2_init() F: E1	jk2_init() Found child <*> in scoreboard slot <*>	
21	Sun Dec 04	notice	jk2_init() F: E1	jk2_init() Found child <*> in scoreboard slot <*>	
22	Sun Dec 04	notice	workerEnv: E2	workerEnv.init() ok <*>	
23	Sun Dec 04	notice	workerEnv: E2	workerEnv.init() ok <*>	
24	Sun Dec 04	notice	workerEnv: E2	workerEnv.init() ok <*>	
25	Sun Dec 04	error	mod_jk chi E3	mod_jk child workerEnv in error state <*>	
26	Sun Dec 04	error	mod_jk chi E3	mod_jk child workerEnv in error state <*>	
27	Sun Dec 04	error	mod_jk chi E3	mod_jk child workerEnv in error state <*>	

Structured CSV output:

Each log line is parsed and mapped to a structured CSV row with fields such as Line ID, Timestamp, Level, Content, Event ID, and Event Template.



Graph page:

The application automatically generates a suite of visualizations, including line plots, pie charts, bar charts, stacked bars, heatmaps, histograms, scatter plots, and area charts.

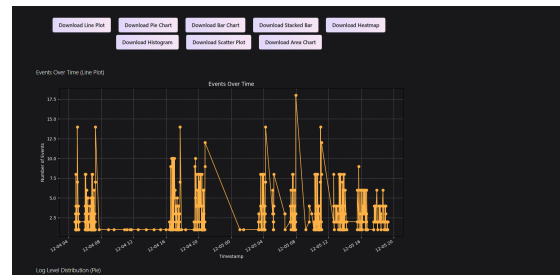
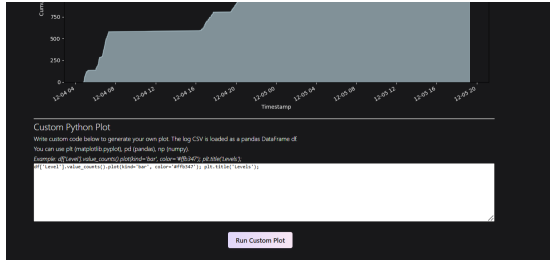


Image download:

Every plot on the graph page can be downloaded as a PNG image.

7.2 Advanced Features

Here we highlight features for power users and error handling, with screenshots for each.



Embedded Python code editor:

For power users, the graph page includes an embedded Python code editor. Users can write and execute custom Python scripts (with access to `df`, `plt`, `pd`, `np`) to generate their own plots, all within the browser.

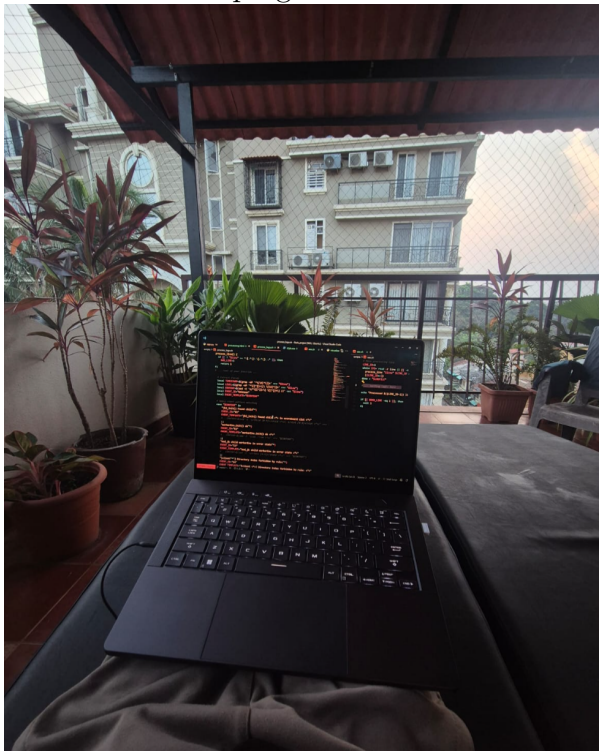
Line ID	Timestamp	Level	Content	Event ID	Event Template
1	Sun Dec 04 04:47:44 2005	notice	worker[enviro] ok,retHttpdCore[worker]2 properties	E2	worker[enviro] ok <*>
2	Sun Dec 04 04:47:44 2005	error	mod_2 child workerEnv in error state 6	E3	mod_2 child workerEnv in error state <*>
3	Sun Dec 04 04:51:08 2005	notice	j2_int0 Found ch4 625 in scoreboard slot 10	E1	j2_int0 Found ch4 <*> in scoreboard slot <*>
4	Sun Dec 04 04:51:09 2005	notice	j2_int0 Found ch4 626 in scoreboard slot 8	E1	j2_int0 Found ch4 <*> in scoreboard slot <*>
5	Sun Dec 04 04:51:09 2005	notice	j2_int0 Found ch4 628 in scoreboard slot 6	E1	j2_int0 Found ch4 <*> in scoreboard slot <*>

Error reporting:

If any log lines are badly formatted, their line numbers are shown in red italics on the display page. These lines are skipped from all tables and visualizations, ensuring the integrity of the analysis.

8 Project Journey

The journey of building this project was as much an adventure as a technical challenge. I started this project while on vacation in Goa which sucks. By day, I roamed the beaches (and tried not to get sand in my laptop); by night, I had to abuse coffee and pull all-nighters, here is a photo of me doing the project in my hotel room balcony at 7am while my friends were sleeping.



Throughout the development, I encountered and overcame a variety of technical and practical challenges:

- **Bash scripting quirks:** Handling CSV quoting, sorting, and line counting correctly (especially when the last line had no newline) was trickier than expected. I learned to use `awk` for robust line counting and to always test edge cases.

- **Real-time progress and error reporting:** Implementing a live progress bar that accurately reflected parsing status (and didn't lie to the user!) required careful coordination between the Bash script, Flask backend, and JavaScript frontend. I also ensured that malformed lines were tracked and reported to the user in a clear, non-intrusive way.
- **Frontend/backend integration:** Making sure that the UI was always in sync with the backend, especially for features like drag-and-drop uploads, progress polling, and dynamic error messages, was a constant exercise in debugging and refinement.
- **Visualization polish:** Tweaking matplotlib and CSS to make the graphs both beautiful and readable took several iterations (and a few aesthetic debates with myself).
- **Keeping it modular and maintainable:** I refactored code and templates repeatedly to ensure that each component (Bash, Flask, JS, HTML/CSS) was clean, modular, and easy to extend.
- **Distractions and time management:** Goa's beaches and Nipun's relentless attempts to lure me away from the keyboard meant I had to become a master of late-night productivity and focus.

In the end, even though because of my train and horrible clab submission thingy I missed the deadline by MILES, I learned that it's possible to build a robust, extensible log analysis suite using only Bash, Python, and a few well-chosen libraries—as long as you have determination, a sense of humor, and a good pair of headphones.

9 Bibliography

- Flask documentation: <https://flask.palletsprojects.com/en/stable/>
- LogPai Apache logs: <https://github.com/logpai/loghub/tree/master/Apache>
- Real Python Flask tutorial: <https://realpython.com/tutorials/flask/>
- Flask Mega-Tutorial: <https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial-p>
- Overleaf report templates: <https://www.overleaf.com/latex/templates/project-report/jpzczmzpsdzwm>
- Bash scripting reference: <https://www.gnu.org/software/bash/manual/>
- Matplotlib documentation: <https://matplotlib.org/stable/contents.html>
- Pandas documentation: <https://pandas.pydata.org/docs/>
- Perplexity AI to find errors in my code: <https://www.perplexity.ai/>

“We strip your logs, sort 'em and show it all off.”